



Resource Sharing Across Teams

Building effective AI infrastructure through shared compute, storage, and data pipelines that balance collaboration and efficiency with proper governance.

Why Share Resources?

AI hardware like GPUs and TPUs represent significant investments with limited availability. Different teams have unique workload patterns:

- Research teams: sporadic, intensive training
- Production: consistent inference loads
- Analytics: periodic batch processing

Utilization

Maximize expensive hardware use across varying workload schedules

Collaboration

Enable cross-team knowledge sharing and model improvement

Cost Savings

Reduce duplicate infrastructure and management overhead

Shared Resources in AI Infrastructure



Compute

Shared GPU/TPU clusters and CPU nodes managed through orchestration platforms like Kubernetes or Slurm



Storage

Distributed object stores (S3, GCS), high-performance file systems, and versioned dataset repositories



Pipelines

Feature stores, preprocessing workflows, and data transformation services that feed multiple models



Model Serving

Centralized inference endpoints with consistent monitoring, scaling and deployment patterns

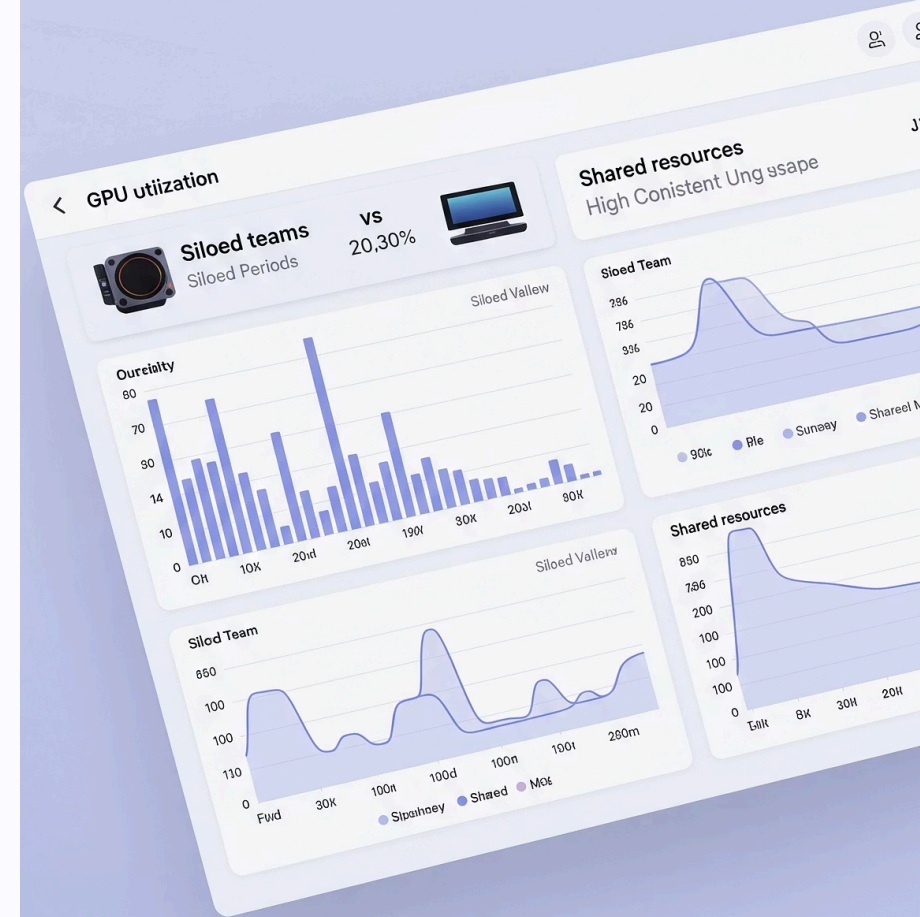
Benefits of Resource Sharing

Technical Benefits

- Eliminate idle GPUs by balancing workloads across time
- Centralize infrastructure management and monitoring
- Implement standardized deployment patterns
- Enable hardware-specific optimizations at scale

Organizational Benefits

- Access to common datasets and pretrained models
- Cross-pollination of techniques between teams
- Unified governance and security policies
- Easier tracking of compute costs and attribution



Risks of Resource Sharing

Noisy Neighbors

High-resource jobs from one team can starve others of compute, causing unpredictable performance and missed deadlines

Data Leakage

Insufficient isolation between teams can lead to unauthorized access to proprietary datasets, models, or business-sensitive information

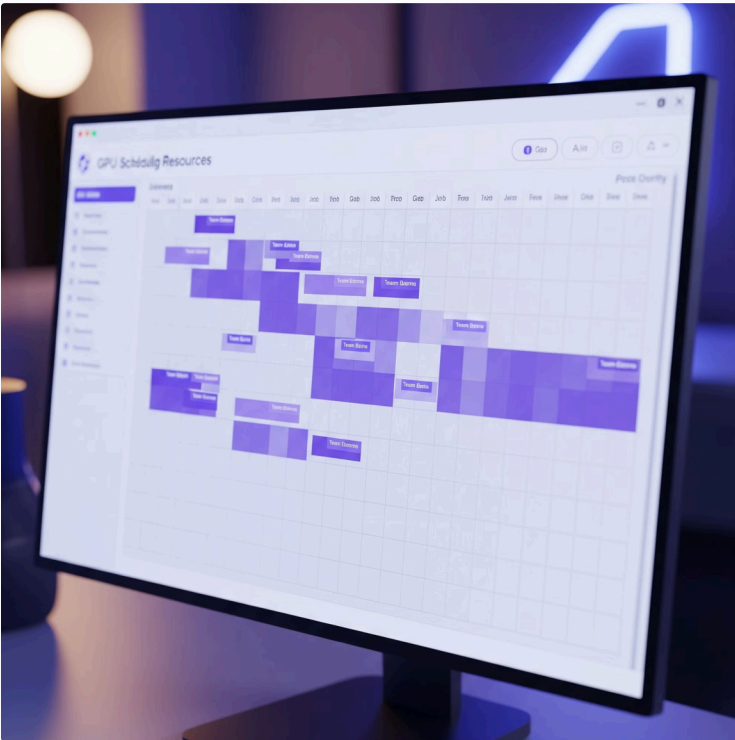
Unfair Cost Distribution

Without proper tracking and chargeback mechanisms, teams with light resource needs may subsidize heavy users

Security Vulnerabilities

Shared infrastructure creates larger attack surfaces and potential privilege escalation paths between teams

Resource Scheduling Strategies



Fair Share

Allocate predefined quotas to each team, ensuring baseline resource availability regardless of demand

Priority Queues

Allow critical production jobs to preempt lower-priority development or experimental workloads

Gang Scheduling

Schedule all components of distributed training jobs simultaneously to prevent resource deadlocks

Elastic Sharing

Dynamically reassign unused capacity from teams under quota to those with waiting jobs

Example: GPU Quota Enforcement



Team Namespaces

Create isolated Kubernetes namespaces for each team with ResourceQuotas limiting GPU requests



Resource Limits

Configure max GPU count per pod and total GPUs per namespace based on team allocation



Node Selection

Use nodeSelectors and taints/tolerations to direct critical workloads to dedicated hardware



Usage Tracking

Implement Prometheus metrics to track actual GPU utilization vs. allocation per team

This approach prevents any single team from monopolizing GPU resources while providing flexibility for priority workloads when needed.

Shared Data & Feature Stores

The Feature Store Advantage

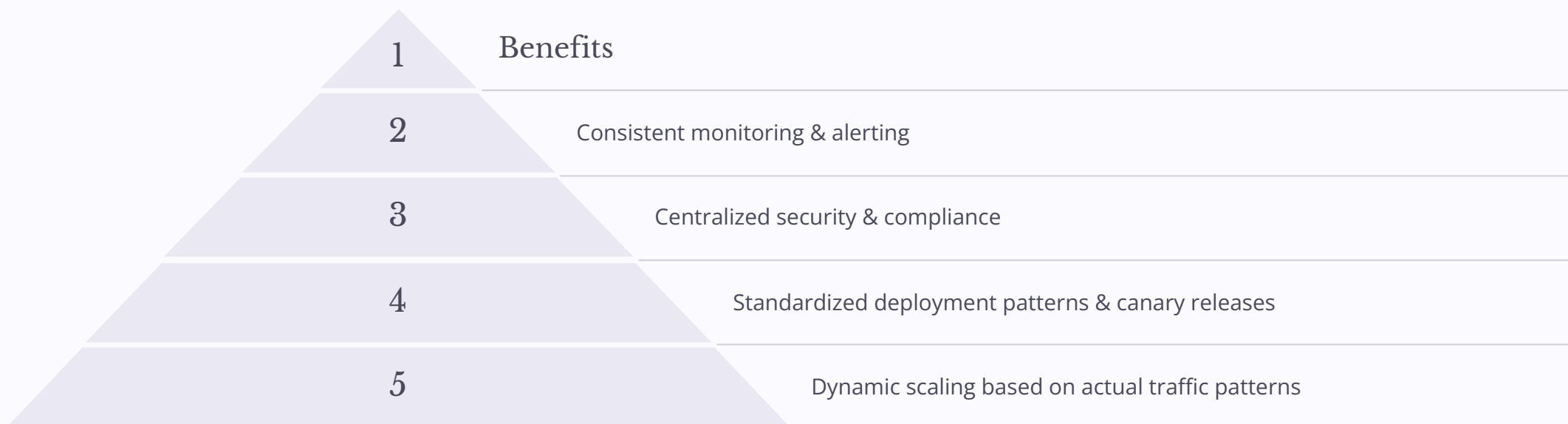
A centralized feature store creates a single source of truth for derived features, enabling:

- Consistent feature transformations across training and inference
- Version control and lineage tracking for reproducibility
- Reduced duplicate computation of common features
- Faster model development through feature reuse
- Standardized data quality monitoring across teams



Required safeguards: Role-based access control, audit logging, data governance policies, and feature-level permissions

Shared Model Serving



Isolation Strategies

Container Isolation: Each model deployed in separate containers with resource limits

Traffic Segmentation: Route requests to specific model instances by team or application

Resource Buffers: Reserve headroom to prevent cascading failures

Performance SLAs: Define and enforce latency expectations per model

Monitoring Shared Resources

Key Metrics to Track

- GPU/TPU utilization per team and job
- Memory consumption patterns
- Storage growth rates by dataset
- Queue wait times for scheduled jobs
- Inference latency across models

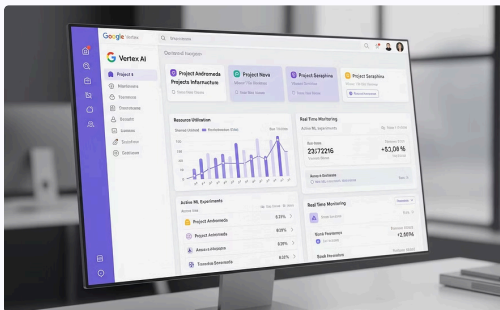
Tooling Stack

Modern monitoring combines multiple specialized tools:

- **Prometheus + Grafana:** Core metrics collection and visualization
- **Kubecost:** Kubernetes cost allocation
- **CloudWatch:** Managed services monitoring
- **OpenTelemetry:** Distributed tracing for pipelines

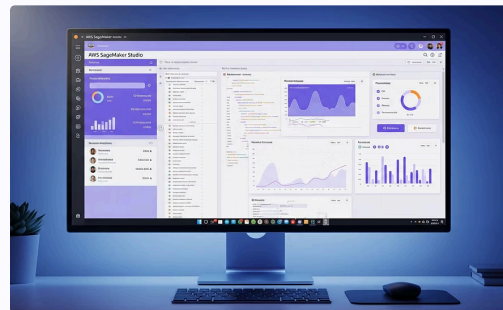


Real-World Examples



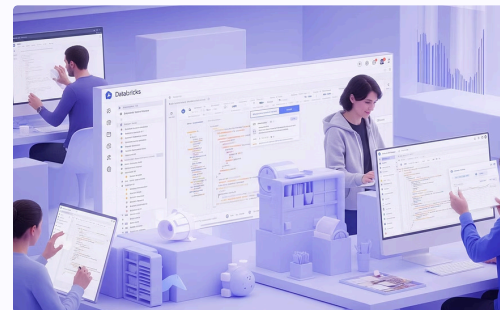
Google Vertex AI

Projects share regional GPU pools with hierarchical quotas and IAM controls



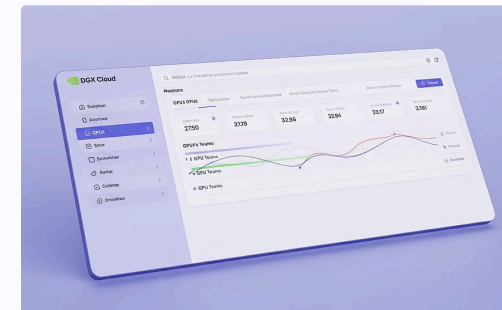
AWS SageMaker Studio

Team workspaces with isolated compute environments and shared model repositories



Databricks

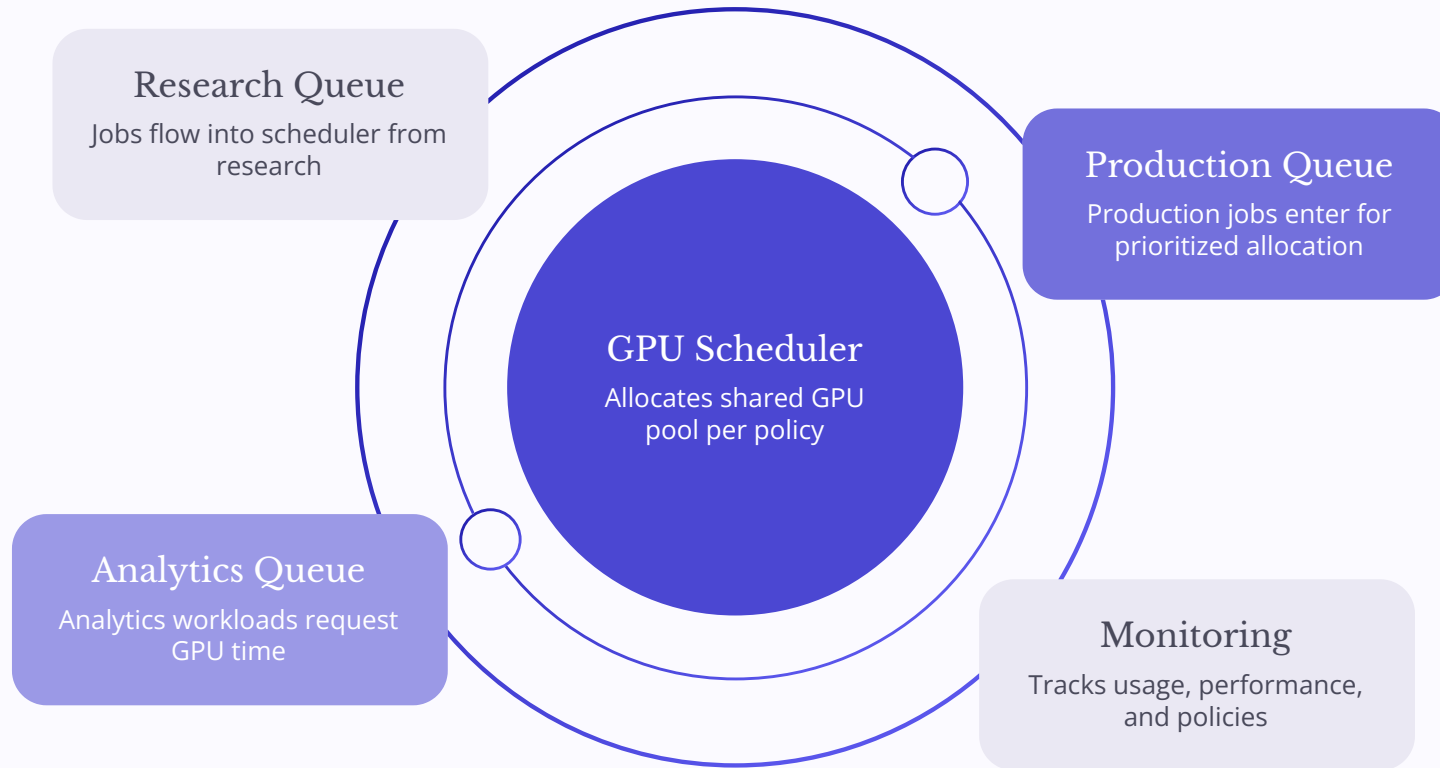
Multi-user notebooks on shared clusters with fine-grained permissions and compute isolation



NVIDIA DGX Cloud

Enterprise-grade pooled GPU resources with workload management across departments

Shared Resource Architecture



A well-architected shared infrastructure requires balance between flexibility and control. The scheduler enforces resource policies while monitoring provides visibility and accountability.

Summary: Balancing Sharing & Governance

Key Benefits

- Improved resource utilization and efficiency
- Cost savings through elimination of siloed infrastructure
- Enhanced collaboration through shared datasets and features
- Standardized deployment and monitoring practices

Critical Safeguards

- Fair scheduling with team quotas and priorities
- Strong isolation between workloads
- Comprehensive monitoring and alerts
- Role-based access control for data and models
- Clear chargeback mechanisms for cost attribution

Next Steps: Conduct resource audit, identify sharing opportunities, implement monitoring first, then gradually migrate to shared infrastructure with appropriate guardrails.